

Experiment No. 3

Aim

Implement non Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm to evaluate average turnaround time and waiting time for a given problem.

Theory

Shortest Job First (SJF) is a CPU scheduling algorithm in which, among all the processes that have arrived and are waiting in the ready queue, the process with the smallest burst time (CPU execution time required) is selected to run next. In its non-preemptive form, once a process is allocated the CPU, it is allowed to run to completion without interruption, even if a process with a shorter burst time arrives afterward - the scheduling decision is only made at the moment the CPU becomes free.

SJF is provably optimal in terms of minimizing the average waiting time among all non-preemptive scheduling algorithms, for a given fixed set of arrival times and burst times, since giving priority to shorter jobs reduces the cumulative time other processes spend waiting behind them. However, SJF requires advance knowledge of each process's burst time, which is generally not known in advance in real operating systems and must be estimated (for example, using exponential averaging of past CPU bursts). It can also lead to starvation of longer processes if shorter processes keep arriving continuously.

For each process, the key performance metrics are computed as follows:

- Completion Time (CT): the time at which a process finishes execution.
- Turnaround Time (TAT): $TAT = \text{Completion Time} - \text{Arrival Time}$, the total time a process spends in the system.
- Waiting Time (WT): $WT = \text{Turnaround Time} - \text{Burst Time}$, the time a process spends waiting in the ready queue.

At each scheduling point, among all processes that have already arrived (Arrival Time $\leq$ current time) but have not yet been executed, the one with the smallest Burst Time is selected next. If no process has arrived yet at the current time, the CPU remains idle until the next process arrives. The average turnaround time and average waiting time are computed by summing these values over all processes and dividing by the total number of processes.

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;

struct Process {
    int id;      // Process ID
    int at;      // Arrival Time
    int bt;      // Burst Time
```

```

    int ct;        // Completion Time
    int tat;       // Turn Around Time
    int wt;        // Waiting Time
};

struct GanttRecord {
    string name;
    int start_time;
    int end_time;
};

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<Process> p(n);
    for (int i = 0; i < n; ++i) {
        p[i].id = i + 1;
        cout << "Enter Arrival Time for process " << p[i].id << ": ";
        cin >> p[i].at;
        cout << "Enter Burst Time for process " << p[i].id << ": ";
        cin >> p[i].bt;
        p[i].ct = p[i].tat = p[i].wt = 0;
    }

    sort(p.begin(), p.end(), [](const Process& a, const Process& b) {
        return a.at < b.at;
    });

    int currentTime = 0;
    int completed = 0;
    vector<bool> done(n, false);

    vector<GanttRecord> gantt;

    while (completed < n) {
        int minBt = INT32_MAX;
        int select = -1;

        for (int i = 0; i < n; ++i) {
            if (!done[i] && p[i].at <= currentTime) {
                if (p[i].bt < minBt) {
                    minBt = p[i].bt;
                    select = i;
                }
            }
        }

        if (select == -1) {

```

```

    int nextArrival = INT32_MAX;
    for (int i = 0; i < n; ++i) {
        if (!done[i] && p[i].at < nextArrival) {
            nextArrival = p[i].at;
        }
    }
    gantt.push_back({"IDLE", currentTime, nextArrival});
    currentTime = nextArrival;
    continue;
}

// Record process execution in Gantt chart
gantt.push_back({"P" + to_string(p[select].id), currentTime, currentTime +
p[select].bt});

// Execute selected process
currentTime += p[select].bt;
p[select].ct = currentTime;
p[select].tat = p[select].ct - p[select].at;
p[select].wt = p[select].tat - p[select].bt;
done[select] = true;
completed++;
}

cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
long long totalTAT = 0, totalWT = 0;
for (int i = 0; i < n; ++i) {
    cout << p[i].id << "\t"
        << p[i].at << "\t"
        << p[i].bt << "\t"
        << p[i].ct << "\t"
        << p[i].tat << "\t"
        << p[i].wt << "\n";
    totalTAT += p[i].tat;
    totalWT += p[i].wt;
}

cout << "\nAverage Turn Around Time: " << (double)totalTAT / n << "\n";
cout << "Average Waiting Time: " << (double)totalWT / n << "\n";

//Output Gantt Chart
cout << "\n--- Gantt Chart ---\n";

//Print top bar
cout << " ";
for (const auto& g : gantt) {
    for (int i = 0; i < g.name.length() + 2; ++i) cout << "-";
    cout << " ";
}
cout << "\n|";

```

```

// Print processes / idle states
for (const auto& g : gantt) {
    cout << " " << g.name << " |";
}
cout << "\n ";

// Print bottom bar
for (const auto& g : gantt) {
    for (int i = 0; i < g.name.length() + 2; ++i) cout << "-";
    cout << " ";
}
cout << "\n";

// Print timelines
cout << gantt[0].start_time;
for (const auto& g : gantt) {
    // Calculate appropriate spacing based on string lengths
    int spaces = g.name.length() + 3 - to_string(g.end_time).length();
    if (spaces < 1) spaces = 1;
    cout << string(spaces, ' ') << g.end_time;
}
cout << "\n";

return 0;
}

```

Output

```

PS C:\Users\lenovo\Desktop\Development\OS Programs> cd "c:\Users\lenovo\Desktop\Development\OS Programs\" ; if
?) { g++ SJF.cpp -o SJF } ; if ($?) { .\SJF }
Enter number of processes: 4
Enter Arrival Time for process 1: 0
Enter Burst Time for process 1: 3
Enter Arrival Time for process 2: 2
Enter Burst Time for process 2: 5
Enter Arrival Time for process 3: 4
Enter Burst Time for process 3: 6
Enter Arrival Time for process 4: 5
Enter Burst Time for process 4: 7

PID    AT    BT    CT    TAT    WT
1       0     3     3     3     0
2       2     5     8     6     1
3       4     6    14    10     4
4       5     7    21    16     9

Average Turn Around Time: 8.75
Average Waiting Time: 3.5

--- Gantt Chart ---
-----
| P1 | P2 | P3 | P4 |
-----
0    3    8   14   21

```

Viva Questions

Q1. What is the difference between preemptive and non-preemptive SJF?

Ans. In non-preemptive SJF, once a process starts executing, it runs to completion without interruption, and the scheduler only picks the next shortest job when the CPU becomes free. In preemptive SJF (also called Shortest Remaining Time First), a running process can be interrupted if a new process arrives with a burst time (or remaining time) shorter than the remaining execution time of the current process.

Q2. Why is SJF considered optimal, and what is its main limitation?

Ans. SJF is optimal because, for a fixed set of processes, it minimizes the average waiting time compared to any other non-preemptive scheduling algorithm — scheduling shorter jobs first reduces the total time other processes spend waiting. Its main limitation is that it requires the burst time of each process to be known in advance, which is generally not possible in real systems and must be predicted or estimated.

Q3. What is starvation, and can it occur in SJF?

Ans. Starvation occurs when a process waits indefinitely because the CPU is repeatedly given to other processes instead. In SJF, starvation can occur for long processes (with large burst times) if a continuous stream of shorter processes keeps arriving, as the scheduler will always prefer the shorter jobs over the long-waiting one.

Q4. How is the next process selected at each scheduling decision point in non-preemptive SJF?

Ans. At each point when the CPU becomes free, the scheduler looks at all processes that have already arrived (Arrival Time $\leq$ current time) but have not yet been executed, and selects the one with the smallest Burst Time among them. If no process has arrived yet, the CPU stays idle until the next process arrives.

Q5. How can the burst time of a process be estimated in a real operating system, since it is not known in advance?

Ans. Since the exact future burst time of a process cannot be known in advance, operating systems typically estimate it using techniques such as exponential averaging (exponential moving average) of the process's previous CPU burst durations, giving more weight to recent bursts to predict the length of the next one.